Switching Costs, Solvers, and Sensitivity

BUS 104: Decision Analysis and Management Science

Danko Turcic

October 27, 2026

UC RIVERSIDE | UNIVERSITY OF CALIFORNIA | School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Lecture Roadmap: One Business Problem, Four Analytical Lenses

Today we follow a single manufacturing case (**Suncrest Artisan Lighting**) through four stages:

1. MILP with Switching Costs (Slides 3–10)

- Why setup/switching costs break LP proportionality.
- Binary switches $y_i \in \{0, 1\}$ and Big-M links $x_i - M_i y_i \leq 0$.
- Why the MILP optimum jumps away from the LP corner!

2. Step-by-Step Python `PuLP 4.0` (Slides 11–16)

- Clean 4-step workflow for continuous and binary variables.
- Simple, exam-ready `print()` loops.

3. Google Sheets `OpenSolver` (Slides 17–20)

- Spreadsheet row/column layout for MILPs.
- Standard form $x_i - M_i y_i \leq 0$ for dragging `=SUMPRODUCT()`.

4. Continuous LP Sensitivity (Slides 21–36)

- Removing switching costs to unlock continuous LP duality.
- **Shadow Prices (`c.pi`)**, **Slacks (`c.slack`)**, and **Reduced Costs (`v.dj`)**.
- Pricing out new candidate products.

The Business Case: Suncrest Artisan Lighting

Context: Suncrest manufactures two luxury brass lamps in daily batches: **Desk Lamps (x_1)** and **Floor Lamps (x_2)**. Producing any positive amount of a lamp requires calibrating specialized jigs (**daily switching/setup cost**).

Department / Metric	Desk (x_1)	Floor (x_2)	Daily Limit
Assembly (hrs/unit)	1 hr	1 hr	$\leq$ 10 hrs
Machining (hrs/unit)	2 hrs	1 hr	$\leq$ 16 hrs
Finishing (hrs/unit)	0 hrs	1 hr	$\leq$ 7 hrs
Gross Margin (c_i)	**\$50**/unit	**\$30**/unit	—
Switching Cost (F_i)	**\$80** if $x_1 > 0$	**\$60** if $x_2 > 0$	—

The Core Managerial Trade-Off

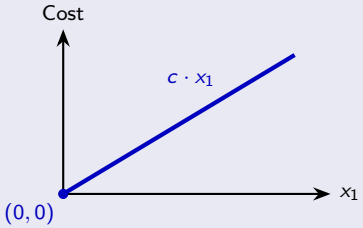
- **Diversify ($x_1 > 0, x_2 > 0$):** Use all three departments fully, but pay **both** switching costs ($\$80 + \$60 = \$140$).
- **Specialize ($x_1 > 0, x_2 = 0$):** Leave some capacity idle, but save the \$60 Floor Lamp setup cost!

Why Switching Costs Break Standard Linear Programming

In a standard LP, cost must be strictly proportional ($C(x_i) = k \cdot x_i$). A switching cost creates a **discontinuous jump at** $x_i = 0$:

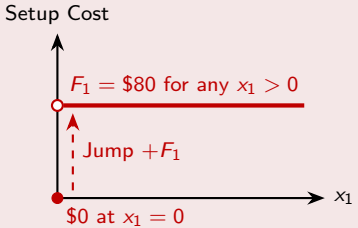
$$\text{Setup Cost}_i(x_i) = \begin{cases} \$0 & \text{if } x_i = 0 \quad \text{(Line } i \text{ stays OFF)} \\ F_i & \text{if } x_i > 0 \quad \text{(Line } i \text{ switched ON)} \end{cases}$$

Graph A: Standard LP (Proportional)



Zero jump at $x_1 = 0$: Continuous LP works.

Graph B: Fixed Switching Cost (MILP)



Discontinuity at 0 requires a **binary switch** $y_i \in \{0, 1\}$.

Step 1: Decision Variables & Net Profit Objective

To model Suncrest's problem as a **Mixed-Integer Linear Program (MILP)**, we define **four decision variables**:

1. Continuous Production Variables

- $x_1 \geq 0$: Number of **Desk Lamps** produced.
- $x_2 \geq 0$: Number of **Floor Lamps** produced.

2. Binary Switch (Setup) Variables

- $y_1 \in \{0, 1\}$: Equals 1 if Desk Lamp line is switched ON; 0 otherwise.
- $y_2 \in \{0, 1\}$: Equals 1 if Floor Lamp line is switched ON; 0 otherwise.

Net Profit Objective Function

$$\max Z = \underbrace{50x_1 + 30x_2}_{\text{Gross Margin}} - \underbrace{(80y_1 + 60y_2)}_{\text{Switching Costs}}$$

Beware the "Solver Cheating" Trap!

Because $-80y_1 - 60y_2$ **penalizes** the objective, an optimizer will automatically set $y_1 = 0$ and $y_2 = 0$ while setting $x_1 > 0, x_2 > 0$ **unless we mathematically force** $y_i = 1$ **whenever** $x_i > 0$**!**

Step 2: The Secret of Big-M Linking Constraints (Two Angles)

How do we enforce the logical rule: **"If $x_i > 0$, then $y_i = 1$"** using linear inequalities?

Angle 1: The Mathematical Gate / Valve

Write the linking constraint for each product i:

$$x_i \leq M_i y_i \quad \Longleftrightarrow \quad \mathbf{x_i - M_i y_i \leq 0}$$

- **Switch OFF ($y_i = 0$):** $x_i \leq M_i(0) = 0 \implies \mathbf{x_i = 0}$. The gate is shut!
- **Switch ON ($y_i = 1$):** $x_i \leq M_i(1) = M_i$. The gate opens up to capacity M_i!

Angle 2: Tightest Valid Physical Upper Bound M_i

What is the largest x_i that could **ever** be feasible given department capacities?

- **Desk Lamps (x_1):** Assembly allows $x_1 \leq 10$; Machining ($2x_1 \leq 16$) allows $x_1 \leq 8$.

$$M_1 = \min\{10, 8\} = \mathbf{8} \implies \mathbf{x_1 - 8y_1 \leq 0}$$

- **Floor Lamps (x_2):** Assembly ≤ 10, Machining ≤ 16, Finishing $x_2 \leq 7$.

$$M_2 = \min\{10, 16, 7\} = \mathbf{7} \implies \mathbf{x_2 - 7y_2 \leq 0}$$

Step 3: Complete MILP Formulation in Standard Form

Writing all variables on the Left-Hand Side (LHS) and constants on the Right-Hand Side (RHS) gives our complete 5-constraint MILP:

Algebraic MILP Model

$$
\begin{aligned}
\max \quad & Z = 50x_1 + 30x_2 - 80y_1 - 60y_2 \\
\text{s.t.} \quad & x_1 + x_2 \leq 10 && \text{(Assembly hrs)} \\
& 2x_1 + x_2 \leq 16 && \text{(Machining hrs)} \\
& x_2 \leq 7 && \text{(Finishing hrs)} \\
& \mathbf{x_1 - 8y_1 \leq 0} && \textbf{(Desk Link, } M_1 = 8\textbf{)} \\
& \mathbf{x_2 - 7y_2 \leq 0} && \textbf{(Floor Link, } M_2 = 7\textbf{)} \\
& x_1, x_2 \geq 0, \quad y_1, y_2 \in \{0, 1\}
\end{aligned}
$$

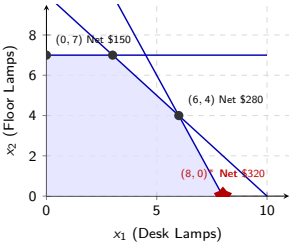
Why Standard Form $x_i - M_i y_i \leq 0$?

- **In Python (`PuLP`):** Either `x1 - 8*y1 <= 0` or `x1 <= 8*y1` is accepted directly.
- **In Google Sheets (`OpenSolver`):** Putting 1 in column x_1 and -8 in column y_1 lets you use **one uniform `=SUMPRODUCT()` formula** across all 5 rows with RHS $= 0$!
- **Model Size:** 2 continuous variables, 2 binary switches, 5 linear constraints.

Step 4: Tracing All 4 Switch Combinations by Hand & Graph

Because $(y_1, y_2) \in \{0,1\}^2$ has only $2^2 = 4$ regimes, we can compare the best corner for each regime:

Switches (y_1, y_2)	**Best** (x_1, x_2)	**Gross** $50x_1 + 30x_2$	**Setup** $80y_1 + 60y_2$	**Net Profit** Z
$(0,0)$	$(0,0)$	\$0	\$0	\$0
$(0,1)$	$(0,7)$	\$210	\$60	\$150
$(1,1)$	$(6,4)$	\$420	\$140	\$280
$(\mathbf{1},\mathbf{0})^*$	$(\mathbf{8},\mathbf{0})^*$	**\$400**	**\$80**	**\$320***



Key Insight: $(8,0)$ Beats $(6,4)$!

Moving from $(8,0)$ to $(6,4)$ adds $+\$20$ in gross margin (\$420 vs. \$400), but triggers a $+\$60$ Floor Lamp setup cost—losing \$40 net!

Interactive Exercise: You Make the Call #1 (Switching Costs)

Scenario: Quick-Change Tooling Redesign on Line 2

Suncrest's industrial engineers redesign the Floor Lamp jig, cutting the Floor Lamp daily switching cost from $F_2 = \$60$ down to $\mathbf{F_2^{new}} = \mathbf{\$15}$ (Desk Lamp setup remains $F_1 = \$80$).

Discuss with a neighbor (2 minutes) and answer all three questions:

1. What is the **new Net Profit** Z if Suncrest operates **both lines** at $(x_1, x_2) = (6, 4)$ with $(y_1, y_2) = (1, 1)$?
2. Should Suncrest now **open both lines at** $(6, 4)$ or **stay Desk-only at** $(8, 0)$ (where Net Profit is \$320)?
3. What is the exact **break-even switching cost** F_2^* at which management is indifferent between $(6, 4)$ and $(8, 0)$?

FORMULA & PARAMETER REFERENCE: $\max Z = 50x_1 + 30x_2 - 80y_1 - F_2 y_2$ | **At** $(8, 0), (1, 0)$**:** Gross $= \$400$, Net $= 400 - 80 = \mathbf{\$320}$ | **At** $(6, 4), (1, 1)$**:** Gross $= 50(6) + 30(4) = \mathbf{\$420}$

Part 2: Step-by-Step MILP Coding in Python (PuLP 4.0)

Every optimization model in PuLP 4.0 follows the exact same **4-Step Blueprint**:

Step 1: Initialize the Model

Create the problem container with `pl.LpProblem("Name", pl.LpMaximize)` (or `pl.LpMinimize`).

Step 2: Define Decision Variables

Use `prob.add_variable()` with `cat="Continuous"` for x_i and `cat="Binary"` for switches y_i.

Step 3: Add Objective & Constraints

Append the objective function and every linear constraint (including Big-M links) using `prob += ...`.

Step 4: Solve & Print Results

Call `res = prob.solve()` and inspect `res.status.name`, `pl.value(prob.objective)`, and `v.varValue`.

PuLP Steps 1 & 2: Initializing the Model & Variables

We first import `pulp as pl`, initialize a maximization problem, and register our continuous production variables (x_1, x_2) alongside our binary switch variables (y_1, y_2):

```
import pulp as pl

# Step 1: Initialize the MILP model
prob = pl.LpProblem("Suncrest_MILP", pl.LpMaximize)

# Step 2a: Continuous production quantities (x1, x2 >= 0)
x1 = prob.add_variable("x1", lowBound=0, cat="Continuous")
x2 = prob.add_variable("x2", lowBound=0, cat="Continuous")

# Step 2b: Binary setup switches (y1, y2 in {0, 1})
y1 = prob.add_variable("y1", cat="Binary")
y2 = prob.add_variable("y2", cat="Binary")
```

Syntax Notes for PuLP 4.0

- Setting `cat="Binary"` automatically restricts $y_i \in \{0, 1\}$ (no need to type `lowBound=0, upBound=1`).
- Setting `lowBound=0` on `x1` and `x2` enforces non-negativity ($x_1 \geq 0, x_2 \geq 0$).

PuLP Step 3a: Adding the Objective & Resource Constraints

Next, we use the += operator to add the net profit objective function (no inequality sign) and the three department hour constraints (with descriptive string names):

```
# Step 3a: Net Profit Objective (Gross Margin minus Switching Costs)
prob += 50 * x1 + 30 * x2 - 80 * y1 - 60 * y2, "Net_Profit"

# Step 3b: Department Capacity Constraints
prob += 1 * x1 + 1 * x2 <= 10, "Assembly"
prob += 2 * x1 + 1 * x2 <= 16, "Machining"
prob += 1 * x2 <= 7, "Finishing"
```

Why Name Every Constraint?

Passing a label like `"Assembly"` after the comma makes model debugging easy with `print(prob)` and lets us identify constraints cleanly when we loop over `prob.constraints()` later!

PuLP Step 3b: Adding the Big-M Linking Constraints

Now we add the two Big-M linking constraints that prevent production unless the corresponding binary switch is turned ON:

```
# Step 3c: Big-M Linking Constraints (Standard Form: x_i - M_i * y_i <= 0)
prob += x1 - 8 * y1 <= 0, "Link_Desk"
prob += x2 - 7 * y2 <= 0, "Link_Floor"
```

Good to Know: Direct Form Works Too!

In PuLP, you can also write:

```
prob += x1 <= 8 * y1, "Link_Desk"
prob += x2 <= 7 * y2, "Link_Floor"
```

PuLP automatically moves `8 * y1` to the left-hand side behind the scenes!

What If You Forgot These Two Lines?

Without `"Link_Desk"` and `"Link_Floor"`, the solver would set `y1 = 0, y2 = 0` and `x1 = 6, x2 = 4`, falsely claiming \$420 profit with zero setup cost!

PuLP Step 4: Solving & Printing the MILP Solution

Finally, we call `prob.solve()` and use simple, readable `print()` statements to inspect the optimal plan:

Python Code (Step 4):

```
# Step 4: Solve the MILP and print results
res = prob.solve()

print("Status:", res.status.name)
print("Optimal Profit =", pl.value(prob.objective))
print("x1 =", x1.varValue)
print("x2 =", x2.varValue)
print("y1 =", y1.varValue)
print("y2 =", y2.varValue)
```

Console Output:

```
Status: Optimal
Optimal Profit = 320.0
x1 = 8.0
x2 = 0.0
y1 = 1.0
y2 = 0.0
```

Exact Match with Hand Solution!

Line 1 is switched ON (`y1 = 1.0`) producing `x1 = 8.0` Desk Lamps; Line 2 stays OFF (`y2 = 0.0, x2 = 0.0`), earning **$320.0** net profit.

Complete All-in-One MILP Script & Loop Printing

Here is the complete 16-line script using a clean `for` loop over `prob.variables()`:

```
import pulp as pl

prob = pl.LpProblem("Suncrest_MILP", pl.LpMaximize)

x1 = prob.add_variable("x1", lowBound=0, cat="Continuous")
x2 = prob.add_variable("x2", lowBound=0, cat="Continuous")
y1 = prob.add_variable("y1", cat="Binary")
y2 = prob.add_variable("y2", cat="Binary")

prob += 50*x1 + 30*x2 - 80*y1 - 60*y2, "Net_Profit"
prob += 1*x1 + 1*x2 <= 10, "Assembly"
prob += 2*x1 + 1*x2 <= 16, "Machining"
prob += 1*x2 <= 7, "Finishing"
prob += x1 - 8*y1 <= 0, "Link_Desk"
prob += x2 - 7*y2 <= 0, "Link_Floor"

res = prob.solve()
print("Status:", res.status.name)
print("Optimal Profit =", pl.value(prob.objective))
for v in prob.variables():
    print(v.name, "Value =", v.varValue)
```

Console Output:

```
Status: Optimal
Optimal Profit = 320.0
x1 Value = 8.0
x2 Value = 0.0
y1 Value = 1.0
y2 Value = 0.0
```

Debugging Tip: `print(prob)`

Adding `print(prob)` before `prob.solve()` prints the entire algebraic formulation to your console so you can verify every sign and coefficient!

Part 3: Designing the Google Sheets Grid (OpenSolver)

In Google Sheets, place all four decision variables (x_1, x_2, y_1, y_2) side-by-side in **Row 4 (B4:E4)**:

	A	B	C	D	E	F	G	H
3	**Variable**	**Desk x_1**	**Floor x_2**	**Switch y_1**	**Switch y_2**	**LHS Formula**	**Sign**	**RHS**
4	**Decision Vars**	**8**	**0**	**1**	**0**			
5	**Unit Profit**	50	30	−80	−60	**320**	`=SUMPRODUCT(B4:E4,B5:E5)`	
6	**Constraints**					**Used (LHS)**		**Limit**
7	Assembly Hrs	1	1	0	0	8	$\leq$	10
8	Machining Hrs	2	1	0	0	16	$\leq$	16
9	Finishing Hrs	0	1	0	0	0	$\leq$	7
10	Link Desk ($M_1 = 8$)	1	0	−8	0	0	$\leq$	0
11	Link Floor ($M_2 = 7$)	0	1	0	−7	0	$\leq$	0

- **Yellow Cells (B4:E4):** Changing variable cells controlled by OpenSolver.
- **Green Cell (F5):** Objective cell `=SUMPRODUCT(B4:E4, B5:E5)`.
- **Blue Cells (F7:F11):** Constraint LHS `=SUMPRODUCT($B$4:$E$4, B7:E7)` dragged down!

Why Standard Form ($x_1 - 8y_1 \leq 0$) Makes Spreadsheets Easy

Notice how Row 10 (**Link Desk**) and Row 11 (**Link Floor**) work in the table on Slide 17:

Method A: Standard Form (**Recommended**)

Rewrite $x_1 \leq 8y_1$ as
$\mathbf{1} \cdot \mathbf{x_1} + \mathbf{0} \cdot \mathbf{x_2} - \mathbf{8} \cdot \mathbf{y_1} + \mathbf{0} \cdot \mathbf{y_2} \leq \mathbf{0}$:

- Put `1` in `B10`, `-8` in `D10`, and `0` in `H10` (RHS).
- Cell `F10` uses the **exact same dragged formula**:

```
=SUMPRODUCT($B$4:$E$4, B10:E10)
```

- You can enter **all 5 constraints as a single block** `F7:F11 <= H7:H11` in OpenSolver!

Method B: Dynamic RHS Formula

Alternatively, keep $x_1 \leq 8y_1$:

- Put `=B4` in the LHS cell `F10`.
- Put `=8*D4` in the RHS cell `H10`.
- Both methods are 100% valid in `OpenSolver`, as long as **formulas live in sheet cells** (never typed inside the OpenSolver dialog box!).

Step-by-Step OpenSolver Configuration in Google Sheets

Open **Extensions** → **OpenSolver** → **Open Sidebar** and fill in the four fields:

OpenSolver Sidebar Settings

1. **Objective Cell:** F5 → Select **Max**.
2. **Variable Cells:** B4:E4 (all 4 decision variables).
3. **Constraints:**
   - F7:F11 <= H7:H11 (All 5 linear constraints)
   - D4:E4 bin (Binary restriction on y_1, y_2)
4. **Unconstrained Variables Non-Negative:** Check ☒ (enforces $x_1, x_2 \geq 0$).
5. **Solver Engine:** CBC (Linear & Integer) → Click **Solve Model**.

Critical Check: Only D4:E4 Are bin!

- Do **not** mark B4:E4 as bin—that would force $x_1, x_2 \in \{0, 1\}$!
- Only the switch cells D4:E4 (y_1, y_2) get the bin rule; B4:C4 (x_1, x_2) remain continuous.

Three Common Midterm Pitfalls in MILP & Spreadsheets

Before we transition to Part 4, make sure you can spot and avoid these three classic errors:

Pitfall 1: Forgetting the `bin` Constraint on y_1, y_2

If y_1 and y_2 are left continuous ($0 \leq y_i \leq 1$), the solver sets $y_1 = x_1/8$ and $y_2 = x_2/7$ (paying only a fraction of the setup cost!). Always verify `cat="Binary"` in PuLP and `bin` in OpenSolver.

Pitfall 2: Picking M_i Too Small (Artificially Clipping Production)

If you accidentally pick $M_1 = 5$ in $x_1 - 5y_1 \leq 0$, then even when $y_1 = 1$, Desk Lamps are capped at $x_1 \leq 5$ instead of 8—cutting off the true optimal solution $(8, 0)$! Always check department capacities to find the true maximum possible output.

Pitfall 3: Forgetting Dollar Signs `$B$4:$E$4` When Dragging `SUMPRODUCT`

Without absolute row references `$4` on the decision variable cells, dragging `=SUMPRODUCT(B4:E4, B7:E7)` down shifts the variable reference to Row 5, Row 6, etc.!

Part 4: Removing Switching Costs → Continuous LP Sensitivity

Now suppose Suncrest's jigs are **already calibrated and bolted down for the month** (switching costs are **zero / sunk**: $F_1 = F_2 = \$0$). We drop the binary variables y_1, y_2 and get a **Pure Continuous LP**:

1. The Operational Angle

- Once setup costs are sunk or eliminated, management wants to maximize daily gross contribution $\max Z = 50x_1 + 30x_2$.
- Now management asks **marginal questions**:
  - *"How much is 1 extra hour of Machining worth?"*
  - *"What if we lose 2 hours of Finishing?"*
  - *"Should we launch a 3rd lamp?"*

2. The Mathematical Angle (Why Drop y_i?)

- **Shadow Prices ($\pi_i = \frac{\partial Z^*}{\partial b_i}$)** and **Reduced Costs (d_j)** come from **Linear Programming Duality**, which requires a convex, continuous feasible region.
- In an MILP with binary jumps, continuous shadow prices are undefined or unreliable. Removing y_i unlocks exact LP sensitivity!

Solving the Pure Continuous LP (Zero Switching Costs)

Without switching costs, let's solve the 3-constraint continuous LP and inspect its geometry:

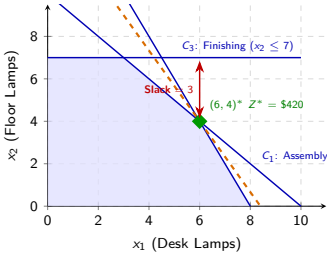
Pure Continuous LP Model

$$\begin{aligned} \max \quad & Z = 50x_1 + 30x_2 \\ \text{s.t.} \quad & x_1 + x_2 \le 10 \quad (C_1\text{: Assembly}) \\ & 2x_1 + x_2 \le 16 \quad (C_2\text{: Machining}) \\ & x_2 \le 7 \quad (C_3\text{: Finishing}) \\ & x_1, x_2 \ge 0 \end{aligned}$$

Optimal LP Corner: $(x_1^*, x_2^*) = (6, 4)$

Intersection of Assembly ($x_1 + x_2 = 10$) and Machining ($2x_1 + x_2 = 16$):

$$x_1^* = 6, \quad x_2^* = 4, \quad \mathbf{Z^* = 50(6) + 30(4) = \$420}$$



Binding vs. Non-Binding Constraints & Slack (`c.slack`)

For any resource constraint $\text{LHS}_i \leq \text{RHS}_i$, its **Slack** is $\text{Slack}_i = \text{RHS}_i - \text{LHS}_i$ at $(x_1^*, x_2^*) = (6, 4)$:

Constraint	LHS Used at $(6,4)$	RHS Capacity	Slack (`c.slack`)	Status
Assembly (C_1)	$1(6) + 1(4) = 10$ hrs	10 hrs	**0.0** hrs	**Binding (Bottleneck)**
Machining (C_2)	$2(6) + 1(4) = 16$ hrs	16 hrs	**0.0** hrs	**Binding (Bottleneck)**
Finishing (C_3)	$0(6) + 1(4) = 4$ hrs	7 hrs	**3.0** hrs	**Non-Binding (Surplus)**

1. Algebraic View

Binding: $\text{LHS}_i = \text{RHS}_i$ ($\text{Slack}_i = 0$).
Non-Binding: $\text{LHS}_i < \text{RHS}_i$ ($\text{Slack}_i > 0$).

2. Geometric View

The optimal corner $(6, 4)$ lies **on** the lines C_1 and C_2, but lies **3 units below** the line C_3.

3. Operational View

Assembly & Machining are 100% utilized bottlenecks; Finishing has **3 idle hours**.

What Is a Shadow Price (`c.pi`)? Two Intuitive Angles

The **Shadow Price** (or **Dual Variable**, denoted π_i or `c.pi` in PuLP) of constraint i is the change in optimal objective value Z^* per $+1$-unit increase in RHS_i (within the allowable range):

$$\Delta Z^* = \pi_i \cdot \Delta b_i$$

Angle 1: Buyer's Max Willingness to Pay

- Suppose an outside vendor offers $+1$ extra hour of department i.
- π_i is the **maximum premium above regular cost** you would pay for that extra hour.
- If an extra hour costs $p < \pi_i$, buying it increases net profit by $(\pi_i - p)$ per hour!

Angle 2: Internal Opportunity Cost

- If a new project consumes 1 hr of department i, pulling it away from current products sacrifices π_i in existing profit.

Complementary Slackness (Golden Rule)

If a constraint has unused capacity ($\text{Slack}_i > 0$), another unit of it is worthless:
$\text{Slack}_\mathbf{i} > \mathbf{0} \implies \pi_\mathbf{i} = \mathbf{\$0}$!

Computing All Three Shadow Prices by Hand

We can verify the shadow prices (π_1, π_2, π_3) at $(6, 4)$ from **two complementary methods**:

Method 1: +1 Hour RHS Perturbation

- **Assembly +1 hr ($x_1 + x_2 = 11, 2x_1 + x_2 = 16$):** New corner is $(5, 6) \implies Z^*_{\text{new}} = 50(5) + 30(6) = \$430$.

$$\pi_1 = 430 - 420 = \mathbf{\$10}/\text{hr}$$

- **Machining +1 hr ($x_1 + x_2 = 10, 2x_1 + x_2 = 17$):** New corner is $(7, 3) \implies Z^*_{\text{new}} = 50(7) + 30(3) = \$440$.

$$\pi_2 = 440 - 420 = \mathbf{\$20}/\text{hr}$$

- **Finishing +1 hr ($\text{Slack}_3 = 3 > 0$):** Corner stays $(6, 4) \implies \pi_3 = \mathbf{\$0}/\text{hr}$.

Method 2: Dual Price Equations

Since $x_1^* > 0$ and $x_2^* > 0$ are both produced, the shadow values of resources inside each lamp must exactly match its unit profit:

$$\begin{aligned} \text{Desk } (x_1): &\quad 1\pi_1 + 2\pi_2 + 0\pi_3 = 50 \\ \text{Floor } (x_2): &\quad 1\pi_1 + 1\pi_2 + 1(0) = 30 \end{aligned}$$

Subtracting the two equations immediately yields $\pi_2 = \mathbf{\$20}$ and $\pi_1 = \mathbf{\$10}$!

Interactive Exercise: You Make the Call #2 (Shadow Prices & Slack)

Two Operational Proposals on Your Desk This Morning

- **Proposal A (Machining Overtime):** A machinist offers to work **2** overtime hours today in the Machining department for an overtime bonus of **\$12**/hour above already-included base wages (assume $+2$ hrs is within the allowable RHS range).
- **Proposal B (Finishing Maintenance):** Facilities needs to shut down the Finishing booth for **2** hours today for filter maintenance (reducing Finishing capacity from 7 hrs to 5 hrs).

Questions (2 minutes):

1. Should you **accept or reject Proposal A**, and what is the **exact net dollar change** in profit?
2. How much profit is lost under **Proposal B** ($\Delta b_3 = -2$ hrs), and why?

FORMULA & PARAMETER REFERENCE: $\Delta Z^* = \pi_i \cdot \Delta b_i$ | **Net Gain of Buying Δb_i hrs at premium p/hr:** $(\pi_i - p) \cdot \Delta b_i$ | **Suncrest LP:** $\pi_1 = \$10$, $\text{Slack}_1 = 0$; $\pi_2 = \$20$, $\text{Slack}_2 = 0$; $\pi_3 = \$0$, $\text{Slack}_3 = 3$ hrs

What Is Reduced Cost (`v.dj`)? Two Intuitive Angles

For any product j with unit profit c_j that consumes a_{ij} hours of each constraint i, its **Reduced Cost** (d_j, accessed via `v.dj` in PuLP) is:

$$\mathbf{d_j} = \underbrace{c_j}_{\text{Unit Profit}} - \underbrace{\sum_i a_{ij}\pi_i}_{\text{Resource Hurdle Rate (Opportunity Cost)}}$$

Angle 1: Net Marginal Profit After Opportunity Cost

- $\sum_i a_{ij}\pi_i$ is the **Hurdle Rate**: the profit you sacrifice elsewhere by diverting resources to build 1 unit of product j.
- If $c_j < \sum_i a_{ij}\pi_i$, then $d_j < 0$: forcing 1 unit of product j **reduces** total profit by $|d_j|$!

Angle 2: Minimum Price Hike to Break Even

- **If $x_j^* > 0$ (Produced): $\mathbf{d_j = 0}$** (c_j exactly equals the resource value inside it).
- **If $x_j^* = 0$ (Not Produced):** $d_j \leq 0$ (in a max LP), and $|d_j|$ is the **minimum unit profit increase** needed before product j becomes worth producing!

Pricing Out a 3rd Candidate Product (x_3: Pendant Lamp)

Suppose R&D proposes a 3rd product: **Pendant Lamps (x_3)**, which sell for a gross margin of $\mathbf{c_3 = \$25}$/unit and require **1** hr of Assembly, **1** hr of Machining, and **1** hr of Finishing.

Step-by-Step Reduced Cost Test for x_3

1. **Resource Recipe** (a_{13}, a_{23}, a_{33})**:** $(1, 1, 1)$ hrs.
2. **Current Shadow Prices** (π_1, π_2, π_3)**:** $(\$10, \$20, \$0)$.
3. **Hurdle Rate (Opportunity Cost of 1 Pendant):**

$$\sum_{i=1}^{3} a_{i3}\pi_i = 1(\$10) + 1(\$20) + 1(\$0) = \mathbf{\$30}$$

4. **Reduced Cost (d_3):**

$$d_3 = c_3 - \sum_{i=1}^{3} a_{i3}\pi_i = \$25 - \$30 = \mathbf{-\$5.00}$$

Verdict on Pendant Lamps (x_3)

- **Optimal Output:** $\mathbf{x_3^* = 0}$ (Do not produce!).
- **Forced Production Penalty:** Forcing $x_3 = 1$ would reduce total profit by $\mathbf{-\$5}$ (from \$420 to \$415).
- **Target Price Hike:** Marketing must raise Pendant gross margin by at least $\mathbf{+\$5}$ (to $c_3 \geq \$30$) before it is worth making!

Continuous LP with x_3 in Python (PuLP 4.0)

Let's code the continuous 3-product LP in PuLP 4.0 to verify all shadow prices, slacks, and reduced costs:

```
import pulp as pl

# 1. Initialize Continuous LP (No switching costs)
prob = pl.LpProblem("Suncrest_LP_Sensitivity", pl.LpMaximize)

# 2. Continuous Decision Variables: Desk (x1), Floor (x2), Pendant (x3)
x1 = prob.add_variable("x1", lowBound=0, cat="Continuous")
x2 = prob.add_variable("x2", lowBound=0, cat="Continuous")
x3 = prob.add_variable("x3", lowBound=0, cat="Continuous")

# 3. Objective Function and Department Constraints
prob += 50 * x1 + 30 * x2 + 25 * x3, "Gross_Profit"

prob += 1 * x1 + 1 * x2 + 1 * x3 <= 10, "Assembly"
prob += 2 * x1 + 1 * x2 + 1 * x3 <= 16, "Machining"
prob += 1 * x2 + 1 * x3 <= 7, "Finishing"

# 4. Solve Continuous LP
res = prob.solve()
```

Extracting c.pi, c.slack, and v.dj in PuLP 4.0

After `res = prob.solve()`, two simple 2-line `for` loops extract the entire sensitivity report:

Python Sensitivity Loops:

```
print("Status:", res.status.name)
print("Optimal Profit =", pl.value(prob.objective))

for c in prob.constraints():
    print(c.name, "Shadow Price =", c.pi, "Slack =", c.slack)

for v in prob.variables():
    print(v.name, "Value =", v.varValue, "Reduced Cost =", v.dj)
```

Exact Console Output:

```
Status: Optimal
Optimal Profit = 420.0
Assembly Shadow Price = 10.0 Slack = -0.0
Machining Shadow Price = 20.0 Slack = -0.0
Finishing Shadow Price = -0.0 Slack = 3.0
x1 Value = 6.0 Reduced Cost = -0.0
x2 Value = 4.0 Reduced Cost = -0.0
x3 Value = 0.0 Reduced Cost = -5.0
```

Note on -0.0 in Solver Output

Floating-point solvers often display zero as `-0.0` (e.g., `Slack = -0.0` or `Shadow Price = -0.0`). Treat `-0.0` as exact mathematical **0**!

100% Match with Our Math!

$\pi = (10, 20, 0)$, Slack $= (0, 0, 3)$, and x_3 has Reduced Cost $= -5.0$.

Interactive Exercise: You Make the Call #3 (Pricing a New Product)

Scenario: Evaluating a Luxury Brass Chandelier (x_4)

Using the PuLP sensitivity output from Slide 31, Suncrest's design team proposes a 4th product, a **Brass Chandelier (x_4)**, requiring:

2 hrs Assembly, **3** hrs Machining, **2** hrs Finishing

Calculate the following (2 minutes):

1. What is the **Resource Hurdle Rate** (total opportunity cost of resources) to build 1 Chandelier?
2. If Marketing prices the Chandelier at a unit gross profit of $\mathbf{c_4} = \$\mathbf{75}$, what is its **Reduced Cost** d_4, and should Suncrest produce it?
3. What if a luxury hotel chain offers a unit gross profit of $\mathbf{c_4} = \$\mathbf{85}$ per Chandelier?

FORMULA & PARAMETER REFERENCE: Hurdle Rate$_j = \sum_i a_{ij}\pi_i$ | Reduced Cost $d_j = c_j - \sum_i a_{ij}\pi_i$
PuLP Shadow Prices: $\pi_{\text{Assembly}} = \$10$, $\pi_{\text{Machining}} = \$20$, $\pi_{\text{Finishing}} = \$0$

Decision Guide: Instant Sensitivity Math vs. Re-Solving the LP

When can you answer an exam or business question **instantly from the current output**, and when must you **re-run `prob.solve()`**?

Answer Instantly (No Re-Solve Needed)

- **Small RHS change on a non-binding constraint:** If capacity drops by less than Slack_i, $\Delta Z^* = \$0$ and x^* is unchanged.
- **Marginal RHS change within allowable range:** $\Delta Z^* = \pi_i \cdot \Delta b_i$.
- **Unprofitable candidate product ($d_j \leq 0$):** Optimal quantity is $x_j^* = 0$ and optimal plan is unchanged.
- **Price hike on an unproduced item ($x_j^* = 0$)** smaller than $|d_j|$: Still $x_j^* = 0$!

Must Re-Solve the Model (`prob.solve()`)

- **Profitable new product ($d_j > 0$):** You know $x_j^* > 0$, but finding the new corner requires re-solving the LP.
- **Large RHS shift beyond the allowable range** (e.g., losing more hours than Slack_i on a non-binding constraint).
- **Simultaneous changes** to multiple objective coefficients and constraints.
- **Any discrete MILP with binary switches $y_i \in \{0, 1\}$!**

Master Synthesis: MILP (Switching Costs) vs. Continuous LP

Comparing our two versions of the Suncrest Artisan Lighting model side-by-side:

Feature	Part 1–3: MILP (Switching Costs)	Part 4: Continuous LP (No Setup Costs)
Decision Variables	$x_1, x_2 \geq 0$ (Cont.) & $y_1, y_2 \in \{0, 1\}$ (Bin.)	$x_1, x_2, x_3 \geq 0$ (Continuous only)
Objective Function	$\max Z = 50x_1 + 30x_2 - 80y_1 - 60y_2$	$\max Z = 50x_1 + 30x_2 + 25x_3$
Big-M Linking Rows	Required: $x_1 - 8y_1 \leq 0$, $x_2 - 7y_2 \leq 0$	None (no binary switches)
Optimal Production	$(\mathbf{x_1^*}, \mathbf{x_2^*}) = (\mathbf{8}, \mathbf{0})$, $(y_1^*, y_2^*) = (1, 0)$	$(\mathbf{x_1^*}, \mathbf{x_2^*}, \mathbf{x_3^*}) = (\mathbf{6}, \mathbf{4}, \mathbf{0})$
Optimal Profit Z^*	**\$320** (\$400 gross −\$80 Desk setup)	**\$420** (gross = net)
Binding Bottlenecks	Machining (16/16 hrs) and Link_Desk	Assembly (10/10) & Machining (16/16)
LP Sensitivity (`pi, dj`)	Not valid due to binary jumps	$\pi = (10, 20, 0)$, Slack $= (0, 0, 3)$, $d_3 = -\$5$

Midterm Exam Review Checklist: Four Core Competencies

Make sure you can confidently execute all four skills from today's lecture:

1. Formulating Switching Costs (MILP)

- Introduce $y_i \in \{0, 1\}$ and subtract $-F_i y_i$ in the profit objective.
- Write tight Big-M linking constraints $x_i - M_i y_i \leq 0$ using physical capacity bounds.

2. Clean `PuLP 4.0` Coding

- Use `prob.add_variable(..., cat="Binary")` or `"Continuous"`.
- Loop over `prob.constraints()` (`c.pi`, `c.slack`) and `prob.variables()` (`v.varValue`, `v.dj`).

3. Google Sheets `OpenSolver` Layout

- Use standard form ($x_i - M_i y_i \leq 0$) so one `=SUMPRODUCT($B$4:$E$4, B7:E7)` formula works for every row.
- Restrict **only** the binary switch cells with `bin`.

4. Continuous LP Sensitivity

- Use π_i (`c.pi`) and Slack_i (`c.slack`) to value overtime/capacity changes ($\Delta Z^* = \pi_i \Delta b_i$).
- Use Reduced Cost $d_j = c_j - \sum_i a_{ij}\pi_i$ (`v.dj`) to price out new products!